# Laser Controller

Software Functional Requirements

By **Gary Twinn**

# Contents

Laser Controller ..... 1
Contents ..... 2
List of Figures ..... 3
List of Tables ..... 4
Overview ..... 5
Query Messages ..... 6
Command Messages ..... 6
Output Messages ..... 8
Web interface ..... 9
Interfaces ..... 10
TTL convertor Schematic ..... 10
Operating system installation ..... 11
Software installation ..... 11
Run Operating System Updates ..... 11
Install PIP3 for Python 3.x installation ..... 11
Nginx installation ..... 11
Install git and gh ..... 11
Configure git and download the valve controller code ..... 12
Install the flask libraries ..... 12
Install the pyserial libraries ..... 12
Nginx Configuration ..... 12
Gunicorn for Python 3.x installation ..... 12

## List of Figures

Figure 1: Web status page ..... 9
Figure 2: Laser controller TTL convertor and RS232 configuration schematic..... 10

## List of Tables

Table 1: Laser status values ..... 8

# Overview

The Laser control software consists of a python (Python Software Foundation 2023) application to control a LaserVall 1PERACKF21 Nd:YAG 808/1064nm laser.

The application is written to use the Flask web interface to provide an HTTP Status page as well as serving requests via the API (Application Programming Interface)

The computer that controls the laser is a Raspberry Pi 4B (Raspberry PI Foundation 2020). The laser unit has two interfaces, an RS232 serial interface for setting the power and reading that status and Transistor-Transistor-Logic (TTL) input for switching on and off the beam.

The Raspberry Pi uses a CH340 USB to RS232 controller to access the RS232 control port.

The laser requires a +5v TTL output to enable the beam, however the Raspberry GPIO consists of a 3.3v Complementary Metal Oxide Semiconductor (CMOS) integrated circuit. In order to step-up the signal voltage an SN74HC125NE 4 buffer integrated circuit (Texas Instruments 1984) was used to convert the CMOS signal to TTL for the Laser control.

The computer is accessed via a RESTful API listening on port 80 for a valid json message.

## Query Messages

Json messages in the following formats are accepted:

Read Laser Status:

```
{
    "item": "laserstatus",
    "command": 1
}
```

The Raspberry Pi will return the laser power level and if the laser if firing or not.

Read Laser Alarm Status:

```
{
    "item": "laseralarm",
    "command": 1
}
```

The Raspberry Pi will query the laser RS323 port and return the status of the laser alarms.

## Command Messages

Json messages in the following formats are accepted:

Switch on laser:

```
{
    "item": "laser",
    "command": "on"
}
```

Switch off laser:

```
{
    "item": "laser",
    "command": "off"
}
```

Set the laser power to nn.n%

```
{
    "setlaserpower": nn.n
}
```

Change the default maximum time the laser can fire to nnn seconds (default is 300)

```
{
    "setlasertimeout": nnn
}
```

# Output Messages

Following a “`laserstatus`” command, the following data is returned:

```
{
    "laser":0,
    "power":45.0
}
```

Following a “`laseralarm`” command, the following data is returned:

```
{
    "laser": 0,
    "power": 45.0,
    "status": 0
}
```

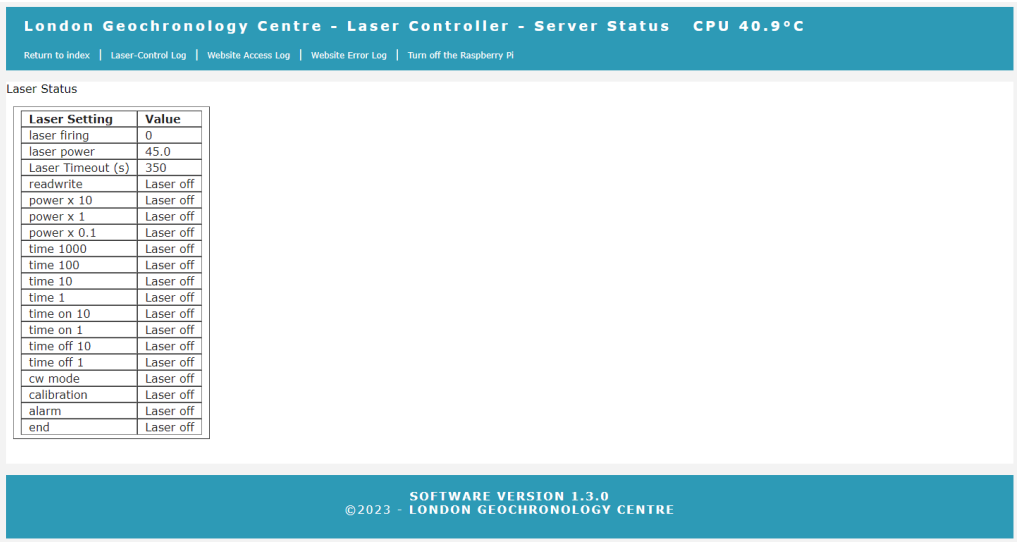
The “`status`” values are

| Value | Alarm Status | Description |
|---|---|---|
| 0 | Powered off | Laser is powered off or the RS323 cable is unplugged. |
| 129 | Emergency active | The laser is poweerd up but the key switch is in position 2 and the enable button has not been pressed. |
| 130 | Wait for start | The key switch is in position 1 and the enable button has not been pressed. |
| 132 | Laser stand by | The key switch is in position 2 and the enable button has not been pressed. |
| 133 | Laser ready | The key switch is in position 2 and the enable button has been pressed. The laser is ready to be used. |
| 141 | Enable not allowed | An error has ocoured such as a power fail while the laser was enabled. |

Table 1: Laser status values

# Web interface

As well as reading and setting the status of the laser via the RESTful API a read-only web interface that can be accessed directly from a browser and will be available to view the laser status, laser settings, application and web server logs.




| Laser Setting | Value |
|---|---|
| laser firing | 0 |
| laser power | 45.0 |
| Laser Timeout (s) | 350 |
| readwrite | Laser off |
| power x 10 | Laser off |
| power x 1 | Laser off |
| power x 0.1 | Laser off |
| time 1000 | Laser off |
| time 100 | Laser off |
| time 10 | Laser off |
| time 1 | Laser off |
| time on 10 | Laser off |
| time on 1 | Laser off |
| time off 10 | Laser off |
| time off 1 | Laser off |
| cw mode | Laser off |
| calibration | Laser off |
| alarm | Laser off |
| end | Laser off |

Figure 1: Web status page

# Interfaces

## TTL convertor Schematic

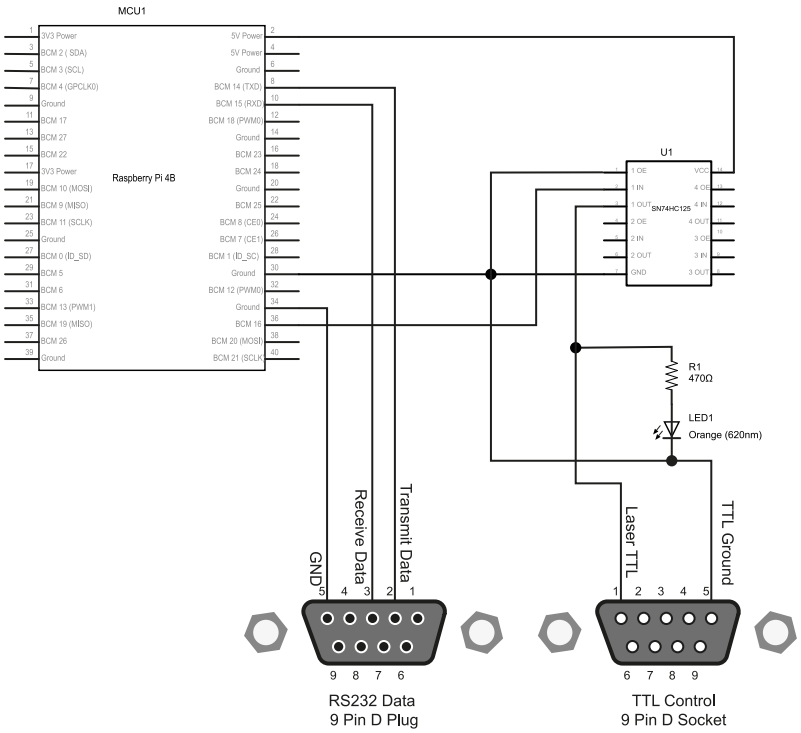



Figure 2: Laser controller TTL convertor and RS232 configuration schematic

# Operating system installation

Use BalenaEtcher to install the latest version of the bookworm-lite 64bit operating system onto a 32Gb MicroMMC card.

Run the `sudo raspi-config` command to:

- enable ssh
- disable Serial
- disable $1^2C$ bus
- set the password on the pi user account to something other than “raspberry”
- change the GPU memory to 16Gb

# Software installation

## Run Operating System Updates

Run `sudo apt update`
Run `sudo apt full-upgrade`

## Install PIP3 for Python 3.x installation

Run `sudo apt install python3-pip`

## Nginx installation

Run `sudo apt install nginx`

## Install git and gh

Run `sudo apt install git`
Run `sudo apt install gh`
Run `gh auth login`

- What account do you want to log into? `GitHub.com`
- What is your preferred protocol for Git operations? `HTTPS`
- Authenticate Git with your GitHub credentials? (Y/n) y
- How would you like to authenticate GitHub CLI? `Paste an authentication token`

## Configure git and download the valve controller code

Create a GitHub folder

Run `mkdir github`

Run `cd github`

Clone the repo

Run `git clone https://github.com/westerlymerlin/UCL-Laser-Controller.git`

Copy the files to the home folder

Run `cp -r ~/github/UCL-Laser-Controller/*  ~/`

Set the execute flag on the scripts

Run `chmod 755 ~/bin/*`

Copy the Raspberry pi config files to the etc folder

Run `sudo cp -r  ~/raspberry-pi/etc/* /etc`

Reboot the Raspberry pi

Run `sudo reboot`

After the reboot a warning banner will appear at ssh logon.

## Install the flask libraries

Run `sudo pip install flask --break-system-packages`

## Install the pyserial libraries

Run `sudo pip install pyserial --break-system-packages`

## Nginx Configuration

Change directory to `/etc/nginx/sites-enabled/`

Run `sudo rm default`

Run `sudo ln -s /etc/nginx/sites-available/lasercontroller`

## Gunicorn for Python 3.x installation

Run `sudo apt install gunicorn3`

Run `sudo systemctl enable gunicorn`

Run `sudo systemctl start gunicorn`

Reboot the Raspberry pi
Run `sudo reboot`

If flask is installed, the python files are in the `/home/pi` directory, gunicorn3 is installed and configured and nginx is installed and configured the web service should be running and the site will be accessible on http://*ip address of the server*

## References


Python Software Foundation (2023) *Python 3 Programming Language*. Online. Available online: https://www.python.org [Accessed March 2023].

Raspberry PI Foundation (2020) *Raspberry PI Model 4B Reference*. Available online: https://www.raspberrypi.org/products/raspberry-pi-4-model-b/ [Accessed July 2020].

Texas Instruments (1984) *SN74HC125NE4 Datasheet*. Available online: https://www.ti.com/lit/ds/symlink/sn74hc125.pdf [Accessed July 2020].